

Moogle Newsroom

Functional Flow & Data Structures

Derived from the live backend: `server/services/news*`, `server/services/newsroom/*`, `server/routes/*newsroom*`, and `shared/schema.ts`.

2026-05-18

1. High-Level Functional Flow

The newsroom is a multi-stage pipeline that turns raw RSS items into Verified Knowledge, which downstream services (debates, podcasts, videos, social posts) consume. Each stage owns one responsibility and writes to a specific table or in-memory queue.

```
RSS feeds (10 sources: OpenAI, DeepMind, MIT TR, VentureBeat,
    TechCrunch, The Verge, HuggingFace, NVIDIA,
    Stanford HAI, arXiv AI)
    | every 30-60 min
    v
[1] INGESTION ..... news-pipeline-service.ts
    | raw rows in news_articles
    v
[2] PROCESSING (AI) ..... gpt-5.5: summary, SEO, classify
    | status='processed'
    v
[3] BREAKING EVAL ..... breaking-news-agent.ts (impact 0-100)
    v
[4] CLUSTERING ..... newsroom/clusteringService.ts
    | EventCluster
    v
[5] CLAIM EXTRACTION ..... newsroom/claimExtractionService.ts
    | ClusterExtraction
    v
[6] VERIFICATION GATE .... newsroom-data-package-service.ts
    | human root-admin -> verified_knowledge
    v
[7] PACKAGING ..... newsroom/newsroomDataPackageBuilder.ts
    | DataPackagePayload
    v
[8] AUTOPILOT DISPATCH ... continuousNewsroomScheduler.ts
                             + autopilotDecisionService.ts (safety)
    |           |           |           |
    v           v           v           v
Debate  Podcast  Avatar Video  Social
pipeline script eng  render svc  distribution
```

2. Stage-by-Stage Reference

#	Stage	Service file	AI model	Tables R/W
1	Ingestion	newsService.ts / news-pipeline-service.ts	—	news_articles (W)
2	Processing	news-pipeline-service.ts	gpt-5.5	news_articles (R/W)
3	Breaking eval	breaking-news-agent.ts	gpt-5.5	news_articles (R/W)
4	Clustering	newsroom/clusteringService.ts	extractor	news_articles (R)
5	Claim extraction	newsroom/claimExtractionService.ts	extractor	in-memory

6	Verification gate	newsroom-data-package-service.ts	— (human)	verified_knowledge, verified_claims, verified_timeline_events (W)
7	Packaging	newsroom/ newsroomDataPackageBuilder.ts	—	verified_* (R)
8	Autopilot dispatch	continuousNewsroomScheduler.ts + autopilotDecisionService.ts	—	downstream tables + state

3. Trust & Safety Gates (in order)

- Deduplication — URL + title hash (news-pipeline-service.ts).
- Classification — AI category (research / product / funding / policy / open-source / breakthrough).
- Impact scoring — breaking-news-agent.ts, 0–100; >80 triggers breaking flag.
- Clustering — Jaccard similarity + time window.
- Claim extraction — lexical extraction + dispute detection.
- TCS (Trust Confidence Score) — trust-engine.ts on extracted claims.
- Content moderation — global middleware (blocked terms, spammers).
- Root-admin verification — hard gate; nothing reaches autopilot/public without verified_knowledge.status='verified'.
- Autopilot decision — re-checks rate limits, dry-run flag, kill-switch.

4. Data Structures

4.1 news_articles

(primary ingestion table)

Column	Type	Notes
id	uuid PK	
title	text	AI-rewritten headline
slug	text unique	drives /api/news/:slug
originalTitle / originalContent	text	RSS source body
summary	text	2-sentence AI summary
content	text	normalized body
seoBlog	text	long-form SEO draft
script	text	short video/podcast script seed
hashtags	text[]	for social distribution
category	text	research/product/funding/policy/open-source/breakthrough
imageUrl	text	hero image
sourceUrl / sourceName / sourceType	text	provenance
status	text	raw -> processed -> published
impactScore	int	0–100
isBreakingNews	bool	impactScore > 80
debateId	uuid FK -> live_debates.id	set when converted
publishedAt	timestamp	

4.2 Engagement tables

news_comments, news_reactions, news_shares.
news_comments.commentType is one of {verification, expert, critic} and is AI-tagged.

4.3 verified_sources

Reliability registry: sourceName, domain, tier (A/B/C), baseScore. Feeds TCS.

4.4 verified_knowledge (immutable canonical story)

Column	Type	Notes
id	uuid PK	
clusterId	text	upstream cluster ref
status	text	draft / verified / retracted
canonicalTitle / canonicalSummary	text	one-true headline + summary
keyFacts	jsonb	array of fact objects
confidence	jsonb	per-fact scores

sourceCoverage	jsonb	sources confirming/disputing
approvedBy	uuid FK -> users	root-admin signer

4.5 verified_claims

Column	Type	Notes
id	uuid PK	
verifiedKnowledgeld	FK	
clusterId	text	
statement	text	single fact claim
verdict	text	confirmed / disputed / unverified
evidence	jsonb	sources, quotes, links

4.6 verified_timeline_events

Append-only history per verifiedKnowledgeld.
eventType is one of {anchor, update, correction}.

4.7 verified_media_references

Image / clip references with rights status.
Consumed by avatar video render.

4.8 podcast_script_packages (downstream join)

Column	Notes
id	PK
debateId	FK -> live_debates.id
sourceArticleId	FK -> news_articles.id
status	draft / approved / rendered
scriptPackage	jsonb (2-min brief + 10-min script)

5. API Surface

5.1 Public

Method	Path	Purpose
GET	/api/news	list published articles
GET	/api/news/:slug	article detail

5.2 Admin (gated)

Method	Path	Gate	Purpose
POST	/api/news/trigger	requireAnyAdminPermission(['content:manage','news:manage'])	manual RSS pipeline run
POST	/api/admin/news-to-debate/generate	requireRootAdmin	convert article -> debate draft

POST	/api/admin/podcast-scripts/generate	requireRootAdmin	convert debate -> podcast package
GET	/api/admin/autopilot/status	requireRootAdmin	24/7 newsroom state
POST	/api/admin/autopilot/start	requireRootAdmin	start continuous scheduler
POST	/api/admin/autopilot/kill-switch	requireRootAdmin	emergency stop

All POSTs sit behind global csrfMiddleware first, then the admin gate. Anonymous POST without a CSRF token returns 403 from CSRF; with a token but no admin session returns 401 from the admin gate.

6. Downstream Join Points

Downstream	Service	Joined via	Result
Debate	news-to-debate-service.ts	news_articles.id -> live_debates.sourceArticleId	draft debate, agents picked by category
Podcast	podcast-script-engine.ts	live_debates.id + news_articles.id	podcast_script_packages row
Video	avatar-video-render-service.ts	scriptPackageId + audioJobId + verified_media_references	render job
Social	social-distribution-service.ts	listens for new knowledgePages / news_articles	per-platform post draft

7. Schedulers

Loop	Owner	Interval	Notes
RSS fetch	newsService.ts	30 min	startup-registered
Pipeline drain	news-pipeline-service.ts	continuous	processes status='raw'
Continuous newsroom	newsroom/continuousNewsroom Scheduler.ts	configurable	autopilot loop; respects kill-switch

8. Failure & Safety Behavior

- Dedupe collision -> row skipped, counter incremented.
- AI call failure -> article stays at status='raw'; retried next pass.
- Verification missing -> no autopilot dispatch, no public exposure.

- Kill-switch ON -> scheduler skips all downstream dispatch but ingestion continues.
- Dry-run mode -> downstream jobs enqueued but marked dryRun=true; no external publish.

Re-generate this document whenever a new stage, table, or downstream consumer is added.